

DECODELABS INDUSTRIAL TRAINING

Project 1: Linux & Command Line Basics

Prepared by:

Onuabueke Martina Chiemezu

Date:

7th july, 2026

1. Introduction

Linux is one of the most widely used operating systems in software development, cloud computing, and DevOps engineering. This project introduces the fundamental Linux command-line interface (CLI) and essential commands used for file management, directory navigation, file permissions, and system administration.

The practical exercises in this report were completed using Ubuntu running on Windows Subsystem for Linux (WSL). Each task demonstrates the use of common Linux commands and explains their real-world applications in DevOps.

2. Project Objectives

The objectives of this project are to:

- Understand the Linux command-line environment.
- Learn basic Linux navigation commands.
- Create and manage files and directories.
- Search for files and text using Linux utilities.
- Understand Linux file permissions.
- Develop confidence using the Linux terminal.
- Build foundational skills required for DevOps engineering.

3. Environment Setup

Ubuntu Terminal

The Ubuntu terminal below was opened through Windows Subsystem for Linux (WSL). The terminal prompt displays important information about the current session, including the logged-in user, the computer name (hostname), the current directory, and the user privilege level.

In my terminal:

- Username: (*martina*)
- Hostname: (*@DESKTOP-RJ7LRA1*)
- Current Directory: Home Directory (*~*)
- User Type: Standard User (*\$*)

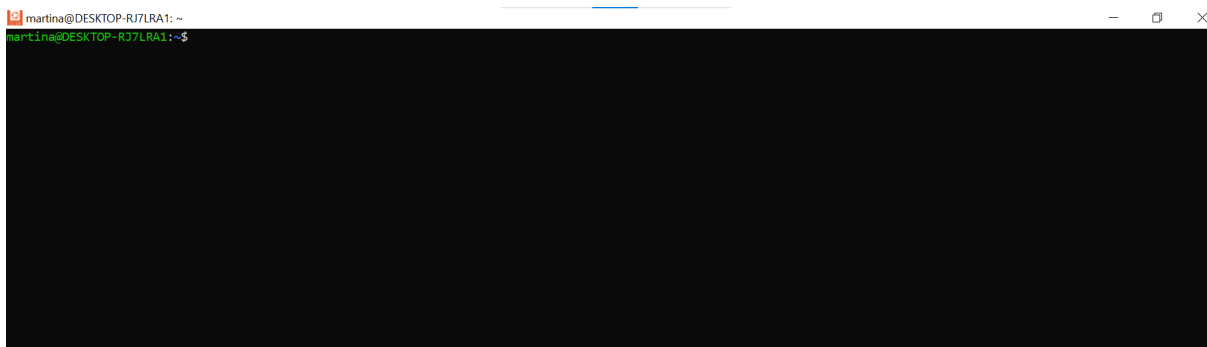


Figure 1: Ubuntu WSL Terminal on Startup

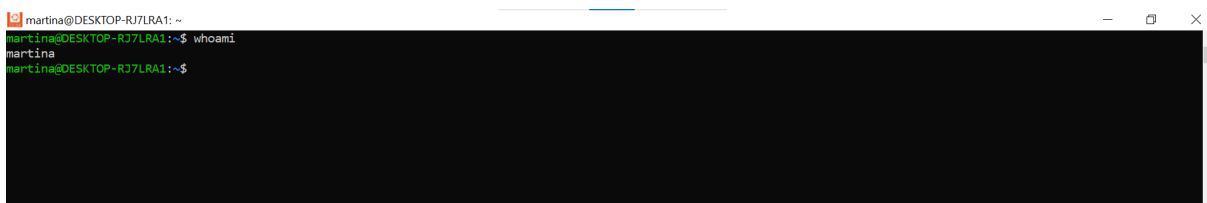
Task 1 — Identify the Current User

Objective

To identify the currently logged-in Linux user.

Command

Whoami



The **whoami** command displays the username of the account currently running the terminal session. This helps users verify their identity before executing commands, especially on systems with multiple user accounts.

DevOps engineers use `whoami` to confirm the current user account before performing administrative tasks or deployments, ensuring they have the appropriate permissions.

Task 2 – Display the Current Working Directory

Objective

To identify the current working directory in the Linux filesystem.

Command

`Pwd`

```
martina@DESKTOP-R37LRA1:~$ pwd
/home/martina
martina@DESKTOP-R37LRA1:~$
```

The `pwd` command displays the absolute path of the current working directory. It helps users know their current location in the Linux filesystem before performing file or directory operations.

DevOps engineers use `pwd` to confirm they are in the correct directory before creating, modifying, or deleting files on servers.

Task 3 – List the Contents of the Current Directory

Objective

To view the files and directories in the current working directory.

Command

`Ls`

```
/home/martina
martina@DESKTOP-R37LRA1:~$ ls
ansible-project  cloud-project-key.pem.pem  getting-started-todo-app  inventory.ini  much-to-do  secure-web-key.pem
cloud-project    devops-directive-kubernetes-course  go  karatu-terraform-project  much-to-do-official  terraform-project
martina@DESKTOP-R37LRA1:~$
```

The `ls` command lists the contents of the current directory. It displays the names of files and directories, allowing users to see what resources are available before navigating or performing operations.

DevOps engineers frequently use `ls` to verify the contents of directories before editing configuration files, deploying applications, or deleting resources.

Task 4 – Display Inode Numbers

Objective

To display the inode number associated with each file and directory in the current working directory.

Command

ls -i

```
martina@DESKTOP-R37LRA1:~$ ls -i
108812 ansible-project      113345 devops-directive-kubernetes-course  108815 inventory.ini          2942 much-to-do-official
108813 cloud-project        110698 getting-started-todo-app             108816 karatu-terraform-project  108817 secure-web-key.pem
108814 cloud-project-key.pem 368456 go                             110426 much-to-do             108818 terraform-project
martina@DESKTOP-R37LRA1:~$
```

The `ls -i` command lists the contents of the current directory together with their inode numbers. An inode is a unique identifier that Linux uses internally to store information about a file or directory.

System administrators and DevOps engineers use inode information when troubleshooting file system issues, investigating hard links, and understanding how Linux stores files.

"Today's Discovery"

I initially thought renaming a file would change its inode. After testing it myself, I discovered that only the filename changes. The inode remains the same because Linux identifies the file by its inode, not by its name.

Task 5 – Display Hidden Files and Directories

Objective

To display all files and directories, including hidden ones, in the current working directory.

Command

ls -a

```
martina@DESKTOP-RJ7LRA1:~$ ls -a
.      .azure      .cache      .kube      .motd_shown  .sudo_as_admin_successful  cloud-project-key.pem.pem  inventory.ini  se
..     .bash_history .config     .landscape .nix-profile .terraform.d  devops-directive-kubernetes-course  karatu-terraform-project  te
.ansible .bash_logout .docker     .lessht   .profile    ansible-project  getting-started-todo-app  much-to-do  te
.aws    .bashrc     .gitconfig  .local    .ssh       cloud-project   go                        much-to-do-official
martina@DESKTOP-RJ7LRA1:~$
```

The `ls -a` command lists all files and directories, including hidden items. In Linux, any file or directory whose name begins with a dot (.) is considered hidden.

DevOps engineers use `ls -a` to view hidden configuration files and directories, such as `.git`, `.ssh`, `.bashrc`, and `.env`, which are commonly used in software development and server administration.

Cd COMMAND

The `cd` (Change Directory) command is used to move from one directory to another in the Linux filesystem. It allows users to navigate through folders and access files stored in different locations.

DevOps engineers use the `cd` command to navigate to project folders, configuration directories, log locations, and application files before performing tasks such as editing files

```
martina@DESKTOP-RJ7LRA1:~/cloud-project$ pwd
/home/martina/cloud-project
martina@DESKTOP-RJ7LRA1:~/cloud-project$ cd ..
martina@DESKTOP-RJ7LRA1:~$ pwd
/home/martina
martina@DESKTOP-RJ7LRA1:~$ cd
martina@DESKTOP-RJ7LRA1:~$ pwd
/home/martina
martina@DESKTOP-RJ7LRA1:~$ cd ..
martina@DESKTOP-RJ7LRA1:/home$ pwd
/home
martina@DESKTOP-RJ7LRA1:/home$ cd
martina@DESKTOP-RJ7LRA1:~$ pwd
/home/martina
martina@DESKTOP-RJ7LRA1:~$
```

MKDIR

The `mkdir` (Make Directory) command is used to create a new directory (folder) in Linux. Directories help organize files into a logical structure, making it easier to manage projects and store related files.

DevOps engineers use the **mkdir** command to create project folders, log directories, backup locations, and configuration directories when setting up applications, servers, or deployment environments.

```
martina@DESKTOP-RJ7LRA1:~$ mkdir linux-practice
martina@DESKTOP-RJ7LRA1:~$ ls
ansible-project  cloud-project-key.pem.pem  getting-started-todo-app  inventory.ini  linux-practice  much-to-do-official  terraform-project
cloud-project    devops-directive-kubernetes-course  go  karatu-terraform-project  much-to-do  secure-web-key.pem
martina@DESKTOP-RJ7LRA1:~$ cd linux-practice
martina@DESKTOP-RJ7LRA1:~/linux-practice$ pwd
/home/martina/linux-practice
martina@DESKTOP-RJ7LRA1:~/linux-practice$
```

A new directory named **linux-practice** is successfully created and can be accessed.

TOUCH

The **touch** command is used to create new empty files or update the timestamp of existing files. It is a simple and efficient way to prepare files for future use.

```
martina@DESKTOP-RJ7LRA1:~/linux-practice$ pwd
/home/martina/linux-practice
martina@DESKTOP-RJ7LRA1:~/linux-practice$ touch notes.txt
martina@DESKTOP-RJ7LRA1:~/linux-practice$ ls
notes.txt
martina@DESKTOP-RJ7LRA1:~/linux-practice$ touch file1.txt file2.txt file3.txt
martina@DESKTOP-RJ7LRA1:~/linux-practice$ ls
file1.txt  file2.txt  file3.txt  notes.txt
martina@DESKTOP-RJ7LRA1:~/linux-practice$ ls -l
total 0
-rw-r--r-- 1 martina martina 0 Jul 7 11:28 file1.txt
-rw-r--r-- 1 martina martina 0 Jul 7 11:28 file2.txt
-rw-r--r-- 1 martina martina 0 Jul 7 11:28 file3.txt
-rw-r--r-- 1 martina martina 0 Jul 7 11:27 notes.txt
martina@DESKTOP-RJ7LRA1:~/linux-practice$
```

CAT

The **cat** command is commonly used to display the contents of files in the terminal. It can also be used to create files and combine multiple files into a single output.

```
martina@DESKTOP-RJ7LRA1:~/linux-practice$ cat notes.txt
My first Linux practical.
I am learning Linux commands.
This is my first internship.martina@DESKTOP-RJ7LRA1:~/linux-practice$
```

CP

The **cp** command is used to create copies of files and directories. It is commonly used to create backups before making changes to important files.

```

This is my first internship.martina@DESKTOP-RJ7LRA1:~/linux-practice$ mkdir backups
martina@DESKTOP-RJ7LRA1:~/linux-practice$ cp notes.txt backups/
martina@DESKTOP-RJ7LRA1:~/linux-practice$ ls backups
notes.txt
martina@DESKTOP-RJ7LRA1:~/linux-practice$ cp -r linux-practice linux-practice-copy
cp: cannot stat 'linux-practice': No such file or directory
martina@DESKTOP-RJ7LRA1:~/linux-practice$ cd ..
martina@DESKTOP-RJ7LRA1:~$ pwd
/home/martina
martina@DESKTOP-RJ7LRA1:~$ cp -r linux-practice linux-practice-copy
martina@DESKTOP-RJ7LRA1:~$

```

MV

DevOps engineers use the **mv** command to organize project files, rename configuration files, archive log files, and move application files between directories during deployments and system maintenance.

```

martina@DESKTOP-RJ7LRA1:~/linux-practice$ mv linux-notes.txt documents/
martina@DESKTOP-RJ7LRA1:~/linux-practice$ ls
backups documents file1.txt file2.txt file3.txt notes_backup.txt
martina@DESKTOP-RJ7LRA1:~/linux-practice$ ls document
ls: cannot access 'document': No such file or directory
martina@DESKTOP-RJ7LRA1:~/linux-practice$ ls documents
linux-notes.txt
martina@DESKTOP-RJ7LRA1:~/linux-practice$ mv documents my-documents
martina@DESKTOP-RJ7LRA1:~/linux-practice$ ls
backups file1.txt file2.txt file3.txt my-documents notes_backup.txt
martina@DESKTOP-RJ7LRA1:~/linux-practice$

```

FIND

The **find** command searches for files and directories. It can search by name or by type, making it useful for locating specific files in Linux.

```

backups file1.txt file2.txt file3.txt my-documents notes_backup.txt
martina@DESKTOP-RJ7LRA1:~/linux-practice$ find . -name "*.txt"
./my-documents/linux-notes.txt
./file2.txt
./backups/notes.txt
./file3.txt
./file1.txt
./notes_backup.txt
martina@DESKTOP-RJ7LRA1:~/linux-practice$ find . -type d
.
./my-documents
./backups

```

GREP

grep searches inside files for specific words or text.

```

This is my first internship.martina@DESKTOP-RJ7LRA1:~/linux-practice$ grep "Linux" my-documents/linux-notes.txt
My first Linux practical.
I am learning Linux commands.
martina@DESKTOP-RJ7LRA1:~/linux-practice$ grep "learning" my-documents/linux-notes.txt
I am learning Linux commands.
martina@DESKTOP-RJ7LRA1:~/linux-practice$ grep "Docker" my-documents/linux-notes.txt
martina@DESKTOP-RJ7LRA1:~/linux-practice$

```

CHMOD

Controls who can read, write, or execute a file.

```
touch script.sh
```

```
ls -l script.sh
```

```
chmod +x script.sh
```

```
ls -l script.sh
```

```
chmod -x script.sh
```

ls -l script.sh

```
martina@DESKTOP-RJ7LRA1:~/linux-practice$ chmod +x script.sh
martina@DESKTOP-RJ7LRA1:~/linux-practice$ ls -l script.sh
-rwxr-xr-x 1 martina martina 0 Jul  7 13:34 script.sh
martina@DESKTOP-RJ7LRA1:~/linux-practice$ chmod -x script.sh
martina@DESKTOP-RJ7LRA1:~/linux-practice$ chmod -x script.sh
martina@DESKTOP-RJ7LRA1:~/linux-practice$ ls -l script.sh
-rw-r--r-- 1 martina martina 0 Jul  7 13:34 script.sh
martina@DESKTOP-RJ7LRA1:~/linux-practice$
```

ECO

The **echo** command is used to display text in the terminal or write text to a file. The **>** operator overwrites a file, while **>>** appends additional content.

```
martina@DESKTOP-RJ7LRA1:~/linux-practice$ echo "Hello Linux"
Hello Linux
martina@DESKTOP-RJ7LRA1:~/linux-practice$ echo "This is my first Echo command." > echo.txt
martina@DESKTOP-RJ7LRA1:~/linux-practice$ cat echo.txt
This is my first Echo command.
martina@DESKTOP-RJ7LRA1:~/linux-practice$ echo "I am learning DevOps." >> echo.txt
martina@DESKTOP-RJ7LRA1:~/linux-practice$ cat echo.txt
This is my first Echo command.
I am learning DevOps.
martina@DESKTOP-RJ7LRA1:~/linux-practice$
```

HISTORY

The **history** command displays previously executed commands, making it easier to repeat or review past work.

```
493 cp notes.txt backups/
494 ls backups
495 cp -r linux-practice linux-practice-copy
496 cd ..
497 pwd
498 cp -r linux-practice linux-practice-copy
499 mv notes.txt linux-notes.txt
500 cd ~/linux-practice
501 ls
502 mv notes.txt linux-notes.txt
503 ls
504 mkdir documents
505 mv linux-notes.txt documents/
506 ls
507 ls document
508 ls documents
509 mv documents my-documents
510 ls
511 find . -name "*.txt"
512 find . -type d
513 find . -type f
514 pwd
515 ls -l
516 find . -type f
517 cat my-documents/linux-notes.txt
518 grep "linux" my-documents/linux-notes.txt
519 grep "learning" my-documents/linux-notes.txt
520 grep "Docker" my-documents/linux-notes.txt
521 touch script.sh
522 ls -l script.sh
523 chmod +x script.sh
524 ls -l script.sh
525 chmod -x script.sh
526 ls -l script.sh
527 echo "Hello Linux"
528 echo "This is my first Echo command." > echo.txt
529 cat echo.txt
530 echo "I am learning DevOps." >> echo.txt
531 history
martina@DESKTOP-RJ7LRA1:~/linux-practice$
```

CLEAR

The **clear** command clears the visible terminal screen without deleting the command history.


```
martina@DESKTOP-RJ7LRA1: ~/linux-practice
martina@DESKTOP-RJ7LRA1:~/linux-practice$
```

MAN

The `man` command opens the manual page for Linux commands. It provides detailed information about command usage, available options, and examples.

```
ls(1)                                User Commands                                ls(1)
NAME
  ls - list directory contents

SYNOPSIS
  ls [OPTION]... [FILE]...

DESCRIPTION
  List information about the FILES (the current directory by default). Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

  Mandatory arguments to long options are mandatory for short options too.

  -a, --all
        do not ignore entries starting with .

  -A, --almost-all
        do not list implied . and ..

  --author
        with -l, print the author of each file

  -b, --escape
        print C-style escapes for nongraphic characters

  --block-size=SIZE
        with -l, scale sizes by SIZE when printing them; e.g., '--block-size=M'; see SIZE format below

  -B, --ignore-backups
        do not list implied entries ending with ~

  -c
        with -lt: sort by, and show, ctime (time of last modification of file status information); with -l: show ctime and sort by name; otherwise: sort by
        ctime, newest first

  -C
        list entries by columns

  --color[=WHEN]
        colorize the output; WHEN can be 'always' (default if omitted), 'auto', or 'never'; more info below

  -d, --directory
        Manual page ls(1) line 1 (press h for help or q to quit)
```

Installing an Additional Linux Utility

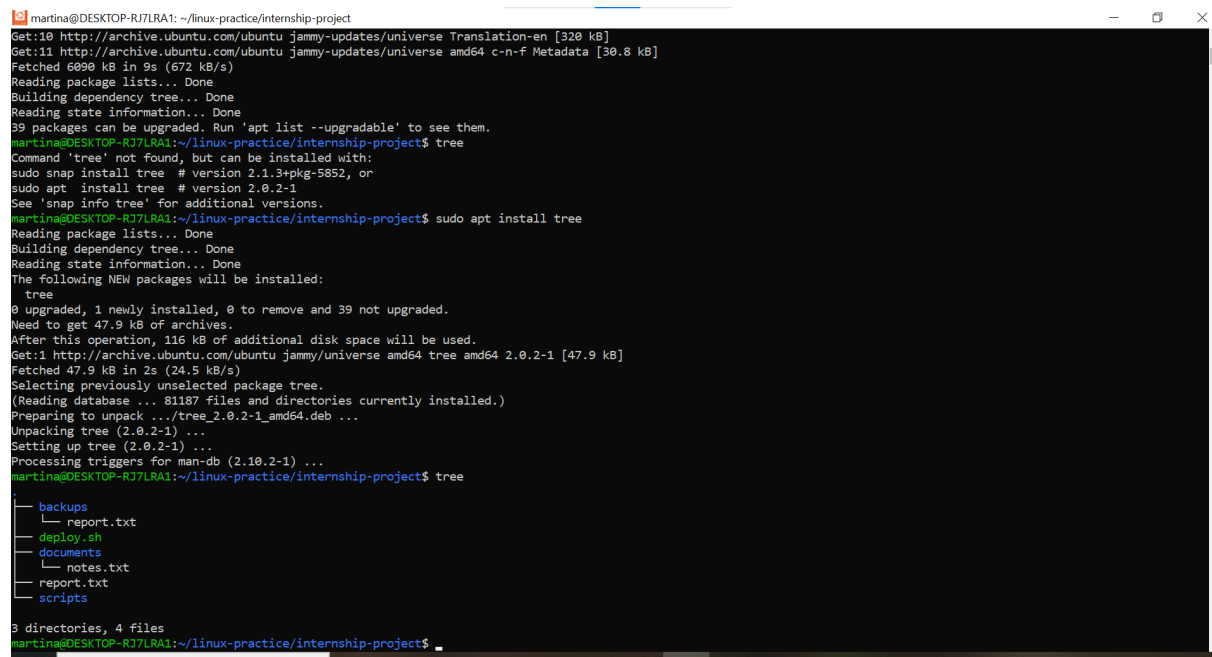
Command Used

`sudo apt update`

`sudo apt install tree`

The **tree** utility was installed using the Ubuntu package manager (**apt**). This tool provides a hierarchical view of directories and files, making it easier to visualize the structure of a project.

System administrators and DevOps engineers use the **tree** command to inspect project structures, verify deployments, and document directory layouts.



```
martina@DESKTOP-RJ7LRA1: ~/linux-practice/internship-project
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [320 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 c-n-f Metadata [30.8 kB]
Fetched 6090 kB in 9s (672 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
39 packages can be upgraded. Run 'apt list --upgradable' to see them.
martina@DESKTOP-RJ7LRA1:~/linux-practice/internship-project$ tree
Command 'tree' not found, but can be installed with:
sudo snap install tree # version 2.1.3+pkg-5852, or
sudo apt install tree # version 2.0.2-1
See 'snap info tree' for additional versions.
martina@DESKTOP-RJ7LRA1:~/linux-practice/internship-project$ sudo apt install tree
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  tree
0 upgraded, 1 newly installed, 0 to remove and 39 not upgraded.
Need to get 47.9 kB of archives.
After this operation, 116 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tree amd64 2.0.2-1 [47.9 kB]
Fetched 47.9 kB in 2s (24.5 kB/s)
Selecting previously unselected package tree.
(Reading database ... 81187 files and directories currently installed.)
Preparing to unpack .../tree_2.0.2-1_amd64.deb ...
Unpacking tree (2.0.2-1) ...
Setting up tree (2.0.2-1) ...
Processing triggers for man-db (2.10.2-1) ...
martina@DESKTOP-RJ7LRA1:~/linux-practice/internship-project$ tree
.
├── backups
│   └── report.txt
├── deploy.sh
├── documents
│   ├── notes.txt
│   └── report.txt
└── scripts

3 directories, 4 files
martina@DESKTOP-RJ7LRA1:~/linux-practice/internship-project$
```